

SignalTap with Verilog Designs

For Quartus® Prime 21.1

1 Introduction

This tutorial explains how to use the SignalTap feature within the Intel® Quartus® Prime software. The SignalTap Embedded Logic Analyzer is a system-level debugging tool that captures and displays signals in circuits designed for implementation in Intel's FPGAs.

Contents:

- Example Circuit
- Enabling the Quartus Prime TalkBack Feature
- Using the SignalTap Logic Analyzer
- Probing the Design Using SignalTap
- Advanced Trigger Options
- Sample Depth and Buffer Acquisition Modes

2 Background

Quartus Prime software includes a system-level debugging tool called SignalTap that can be used to capture and display signals in real time in any FPGA design.

During this tutorial, the reader will learn about:

- Probing signals using the SignalTap software
- Setting up triggers to specify when data is to be captured

This tutorial is aimed at the reader who wishes to probe signals in circuits defined using the Verilog hardware description language. An equivalent tutorial is available for the reader who prefers the VHDL language.

The reader is expected to have access to a computer that has Quartus Prime software installed. The detailed examples in the tutorial were obtained using Quartus Prime version 21.1, but other versions of the software can also be used.

Note:

There are no red LEDs on a DE0-Nano board. All procedures using red LEDs in this tutorial are to be completed on the DE0-Nano board using green LEDs instead. If you are doing this tutorial on a DE0-Nano board, replace *LEDR* with *LED* below. Additionally, the DE0-Nano is limited to 2 keys. If you are doing this tutorial on a DE0-Nano, replace all occurrences of *[3:0]* with *[1:0]* below.

3 Example Circuit

As an example, we will use the key circuit implemented in Verilog in Figure 1. This circuit simply connects the first 4 keys on a DE-series board to the first 4 red LEDs on the board. It does so at the positive edge of the clock (CLOCK_50) by loading the values of the keys into a register whose output is connected directly to the red LEDs.

```
// Top-level module
module keys (KEY, CLOCK_50, LEDR);
    input [3:0] KEY;
    input CLOCK_50;
    output reg [3:0] LEDR;

    always @(posedge CLOCK_50)
        LEDR [3:0] <= KEY [3:0];
endmodule
```

Figure 1. The key circuit implemented in Verilog code

Implement this circuit as follows:

- Create a project *keys*.
- Include a file *keys.v*, which corresponds to Figure 1, in the project.
- Select the correct device that is associated with the DE-series board. A list of device names for the DE-series boards can be found in Table 1.
- Import the relevant qsf file. For example, for a DE1-SoC board, this file is called *DE1_SoC.qsf* and can be imported by clicking **Assignments > Import Assignments**. For convenience, the qsf files are hosted on the Intel FPGA University Program's [website](#). Simply navigate to the materials section of your DE-series board's page. The node names used in the sample circuit correspond to the names used in these files.
- Compile the design.

Board	Device Name
DE0-CV	Cyclone® V 5CEBA4F23C7
DE0-Nano	Cyclone® IVE EP4CE22F17C6
DE0-Nano-SoC	Cyclone® V SoC 5CSEMA4U23C6
DE1-SoC	Cyclone® V SoC 5CSEMA5F31C6
DE2-115	Cyclone® IVE EP4CE115F29C7
DE10-Lite	Max® 10 10M50DAF484C7G
DE10-Standard	Cyclone® V SoC 5CSXFC6D6F31C6
DE10-Nano	Cyclone® V SE 5CSEBA6U2317

Table 1. DE-series FPGA device names

4 Using the SignalTap software

In the first part of the tutorial, we are going to set up the SignalTap Logic Analyzer to probe the values of the 4 LED keys. We will also set up the circuit to trigger when the first key (LED[0]) is low.

1. Open the SignalTap window by selecting **File > New**, which gives the window shown in Figure 2. Choose **SignalTap Logic Analyzer File** and click OK.

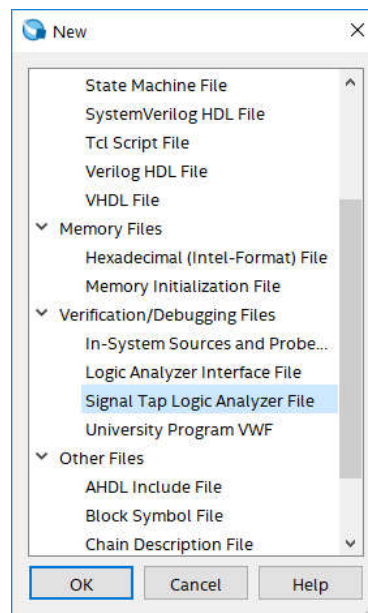


Figure 2. Need to prepare a new file.

2. The SignalTap window with the **Setup** tab selected is depicted in Figure 3. Save the file under the name *keys.stp*. In the dialog box that follows (Figure 4), click **OK**. For the dialog "Do you want to enable SignalTap file '*keys.stp*' for the current project?" click **Yes** (Figure 5). The file *keys.stp* is now the SignalTap file associated with the project.

Note: If you want to disable this file from the project, or to disable SignalTap from the project, go to **Assignments > Settings**. In the category list, select **SignalTap Logic Analyzer**, bringing up the window in Figure 6. To turn off the analyzer, uncheck **Enable SignalTap Logic Analyzer**. It is possible to have multiple SignalTap files for a given project, but only one of them can be enabled at a time. Having multiple SignalTap files might be useful if the project is very large and different sections of the project need to be probed. To create a new SignalTap file for a project, simply follow Steps 1 and 2 again and give the new file a different name. To change the SignalTap file associated with the project, in the **SignalTap File name** box browse for the file wanted, click **Open**, and then click **OK**. For this tutorial we want to leave SignalTap enabled and we want the SignalTap File name to be *keys.stp*. Make sure this is the case and click **OK** to leave the settings window.

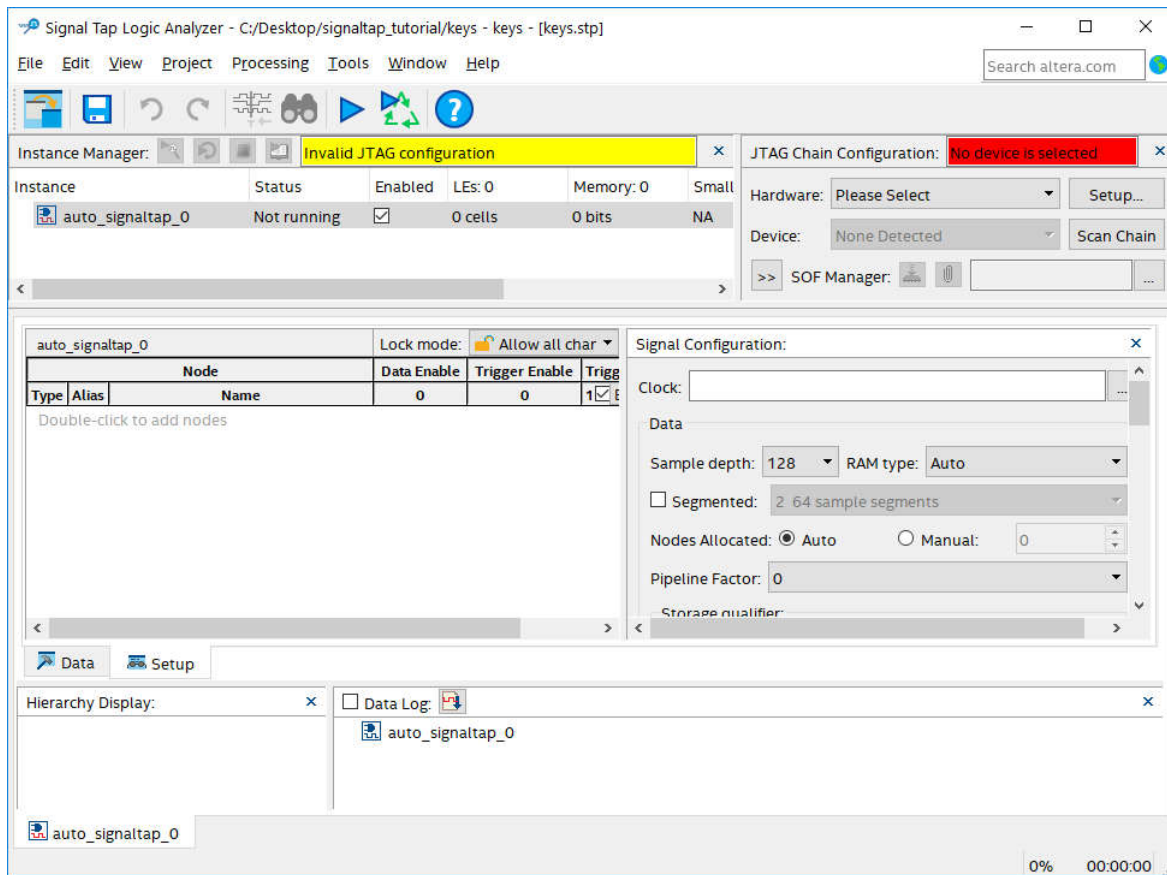


Figure 3. The SignalTap window.

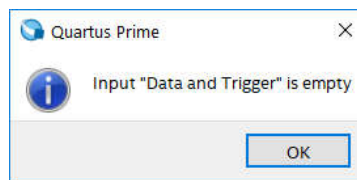


Figure 4. Click OK to this dialog.

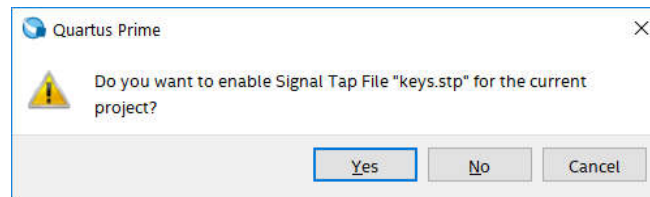


Figure 5. Click Yes to this dialog.

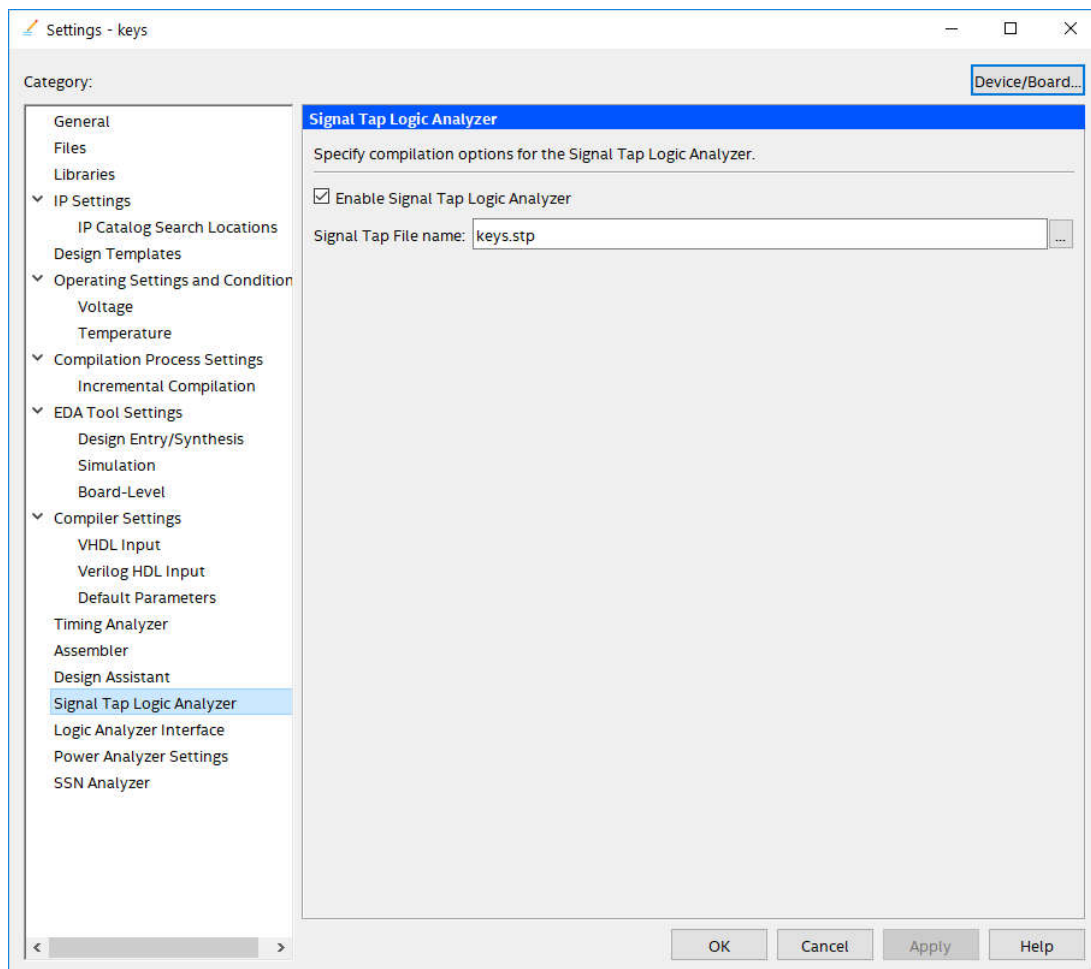


Figure 6. The SignalTap Settings window.

3. We now need to add the nodes in the project that we wish to probe. In the Setup tab of the SignalTap window, double-click in the area labeled Double-click to add nodes, bringing up the Node Finder window shown in Figure 7. Click on  or  to show or hide more search options. For the Filter field, select **SignalTap: pre-synthesis**, and for the Look in field select **|keys|**. Click List. This will now display all the nodes that can

be probed in the project. Highlight KEY[0] to KEY[3], and then click the  button to add the keys to be probed. Click **Insert** to insert the selected nodes, then **Close** to close the Node Finder window.

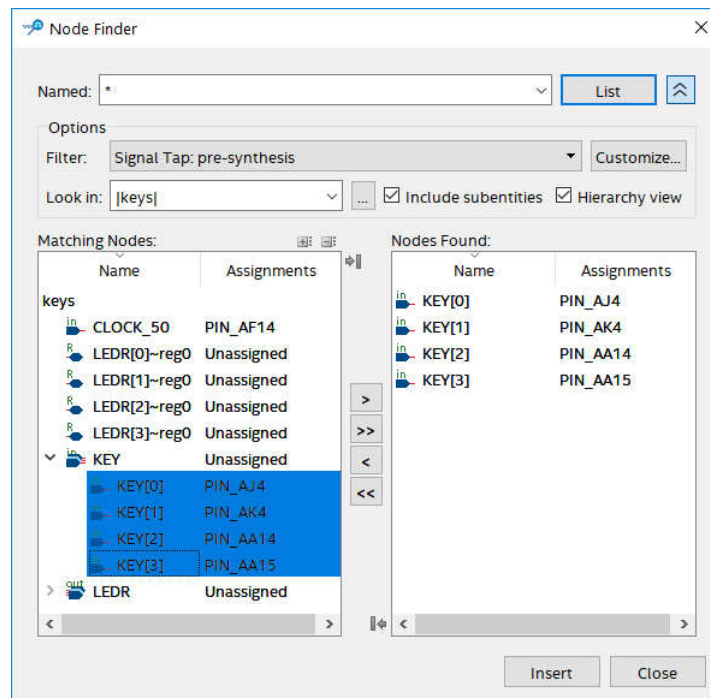


Figure 7. Adding nodes in the Node Finder window on a DE-series board.

4. Before the SignalTap analyzer can work, we need to specify what clock is going to run the SignalTap module that will be instantiated within our design. To do this, in the Clock box of the Signal Configuration pane of the SignalTap window, click , which will again bring up the Node Finder window. Select **List** to display all the nodes that can be added as the clock, and then double-click **CLOCK_50**, which results in the image shown in Figure 8. Click **OK**.

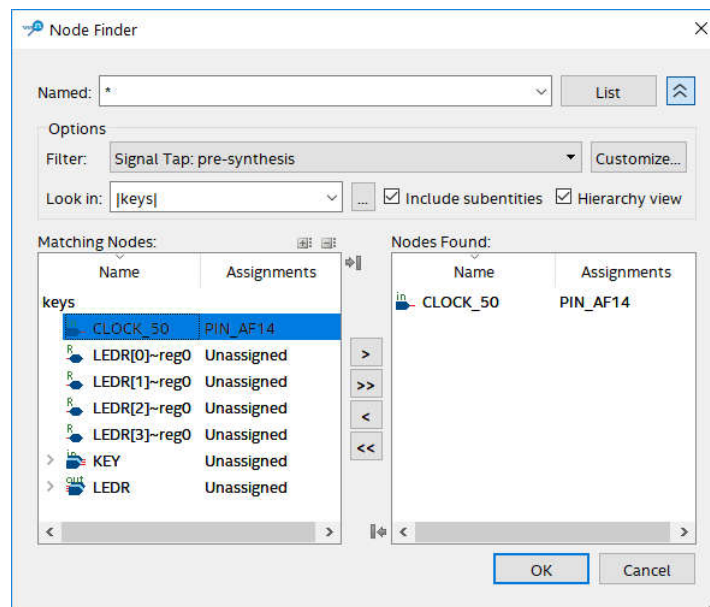


Figure 8. Setting CLOCK_50 as the clock for the SignalTap instance on a DE-series board.

- With the **Setup** tab of the SignalTap window selected, select the checkbox in the Trigger Conditions column. In the dropdown menu at the top of this column, select **Basic AND**. Right-click on the Trigger Conditions cell corresponding to the node **KEY[0]** and select **Low**. Now, the trigger for running the Logic Analyzer will be when the first key on the DE-series board is pressed, as shown in Figure 9. Note that you can right-click on the Trigger Conditions cell of any of the nodes being probed and select the trigger condition from a number of choices. The actual trigger condition will be true when the logical AND of all these conditions is satisfied. For now, just keep the trigger condition as **KEY[0]** set to low and the others set to their default value, **Don't Care**.

Node			Data Enable	Trigger Enable	Trigger Conditions
Type	Alias	Name	4	4	1 Basic AND
in		KEY[0]	☒	☒	0
in		KEY[1]	☒	☒	
in		KEY[2]	☒	☒	
in		KEY[3]	☒	☒	

Figure 9. Setting the trigger conditions.

- For SignalTap to work, we need to properly set up the hardware. First, make sure the DE-series board is plugged in and turned on. In the Hardware section of the SignalTap window, located in the top right corner, click **Setup...**, bringing up the window in Figure 10. Double click DE-SoC in the Available Hardware Items menu, then click **Close**. If you are using a DE0-CV, DE0-Nano, DE2-115, or the DE10-Lite, you will select USB-Blaster from the Available Hardware Items menu.

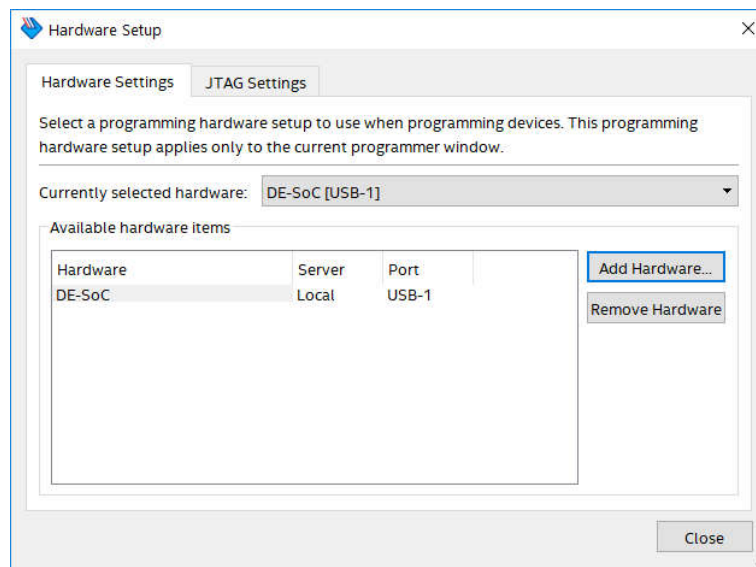


Figure 10. Setting up hardware.

7. In the Device section of the main SignalTap window, select the device that corresponds to the FPGA on your DE-series board. Do not select the SOCVHPS device as this corresponds to the ARM Cortex-A9* processor. If you are using the DE0-CV, DE0-Nano, DE2-115, or DE10-Lite there should be only one device that is selectable.
8. The last step in instantiating SignalTap in your design is to compile the design. In the main Quartus Prime window, select **Processing > Start Compilation** and indicate that you want to save the changes to the file by clicking **Yes**. After compilation, go to **Tools > Programmer** and load the project onto the DE-series board. You may also need to select the corresponding SOF file in the SOF Manager box in the main SignalTap window.

5 Probing the Design Using SignalTap

Now that the project with SignalTap instantiated has been loaded onto the DE-series board, we can probe the nodes as we would with an external logic analyzer.

1. On the DE-series board, first ensure that none of the keys (0-3) is being pressed. We will try to probe the values of these keys once key 0 is pressed.
2. In the SignalTap window, select **Processing > Run Analysis** or click the  icon. You should get a screen similar to Figure 11. Note that the status column of the SignalTap Instance Manager pane says "Waiting for trigger." This is because the trigger condition (Key 0 being low) has not yet been met.

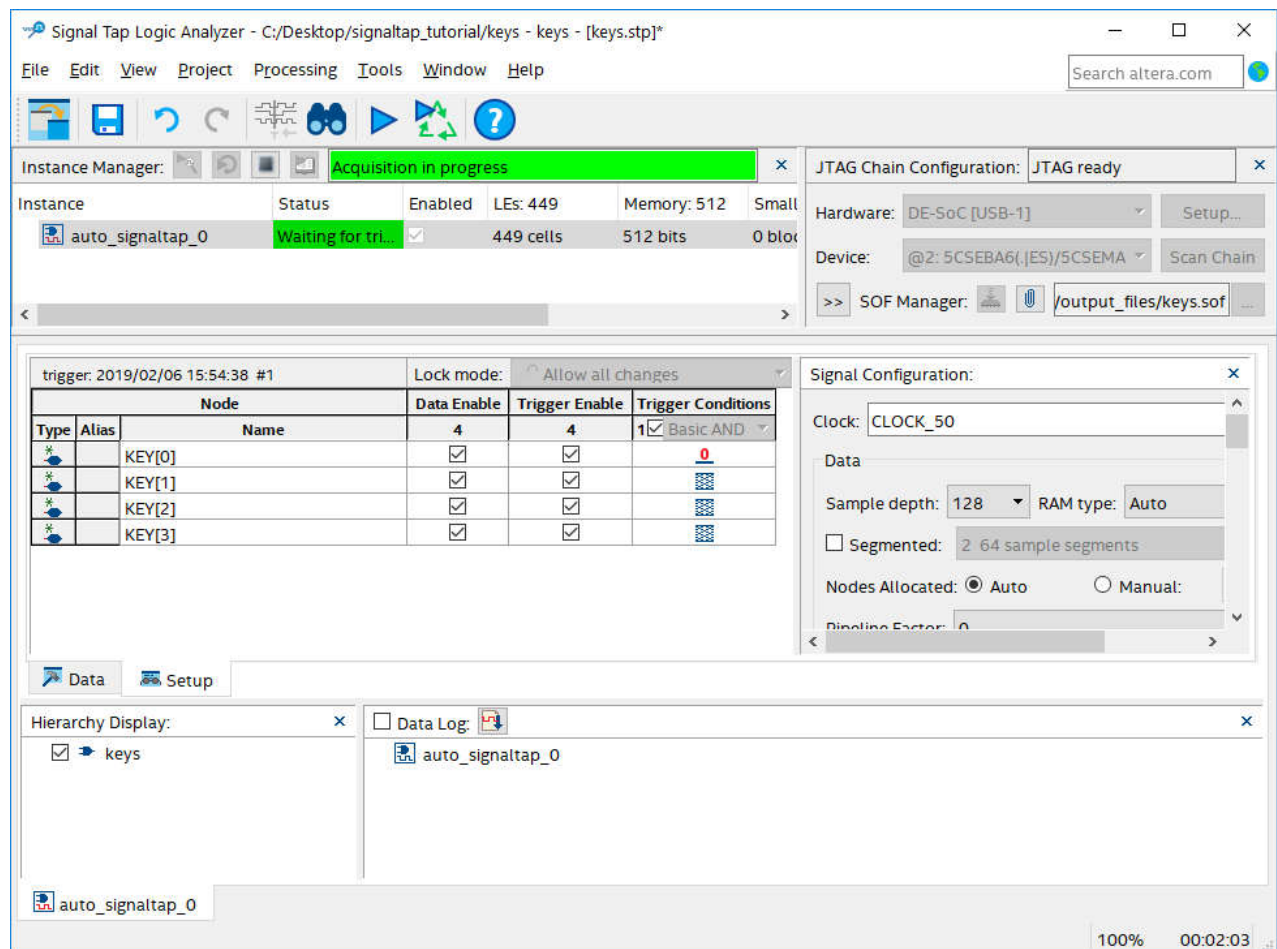


Figure 11. SignalTap window on a DE-series board after Run Analysis has been clicked.

- Now, to observe the trigger feature of the Logic Analyzer, click on the **Data** tab of the SignalTap Window and then press and hold Key 0 on the DE-series board. The data window of the SignalTap window should display the image in Figure 12. Note that this window shows the data levels of the 4 nodes before and after the trigger condition was met. As an exercise, unpress Key 0 then click **Run Analysis** again. Hold down any of Keys 1-3, then press Key 0. When Key 0 is pressed, you will see that the values of Keys 1-3 displayed on the SignalTap Logic Analyzer match what is being pressed on the board.

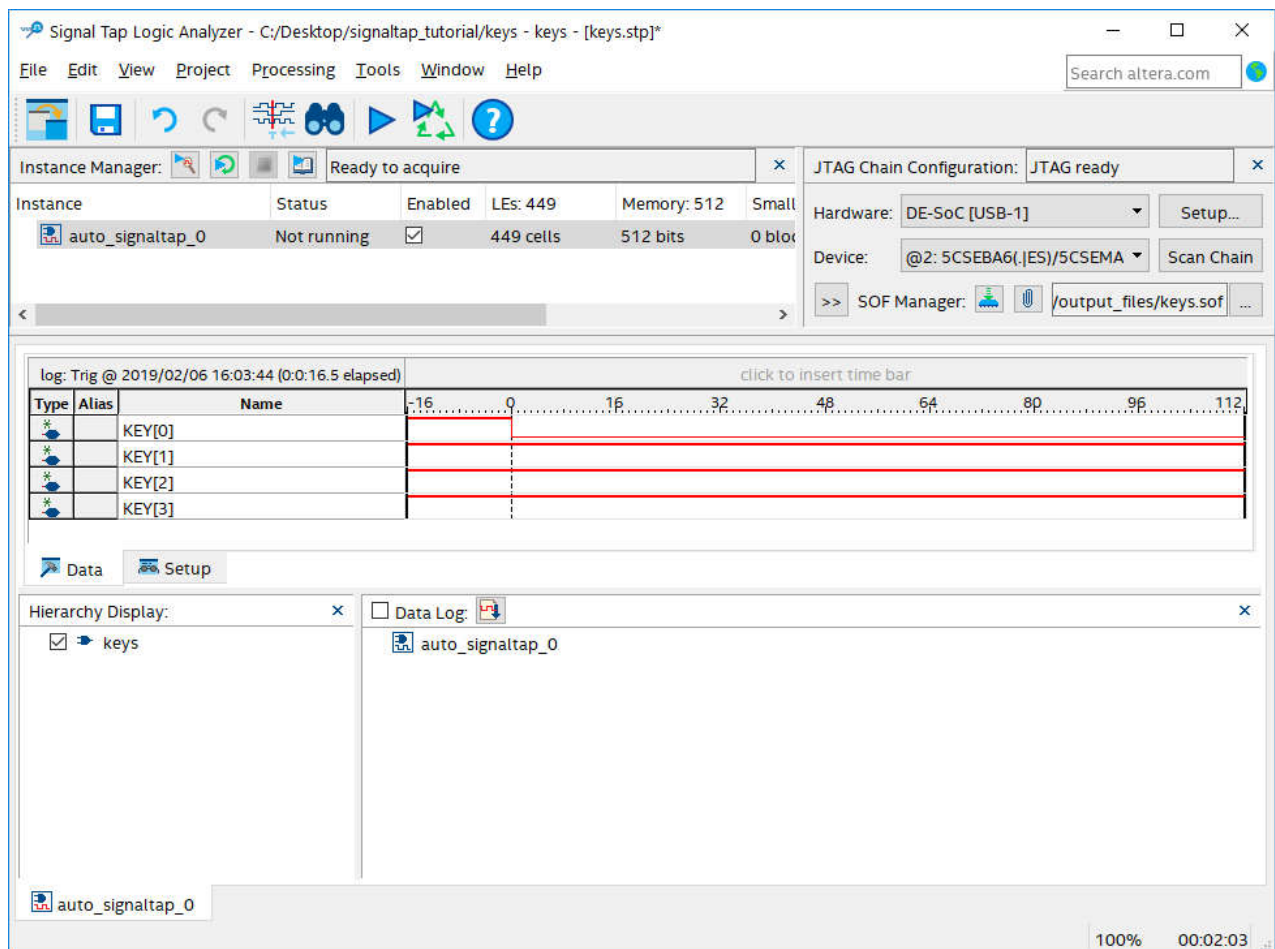


Figure 12. Graphical display of values after trigger condition is met.

6 Advanced Trigger Options

Sometimes in a design you may want to have a more complicated triggering condition than SignalTap's basic triggering controls allow. The following section describes how to have multiple trigger levels.

6.1 Multiple Trigger Levels

In this section, we will set up the analyzer to trigger when there is a positive edge from Key 0, Key 1, Key 2, and then Key 3, in that order.

1. Click the Setup tab of the SignalTap window.

- In the Signal Configuration pane, select 4 from Trigger Conditions dropdown menu as in Figure 13 (you may have to scroll down in the Signal Configuration pane to see this menu). This modifies the node list window by creating three new Trigger Conditions columns.

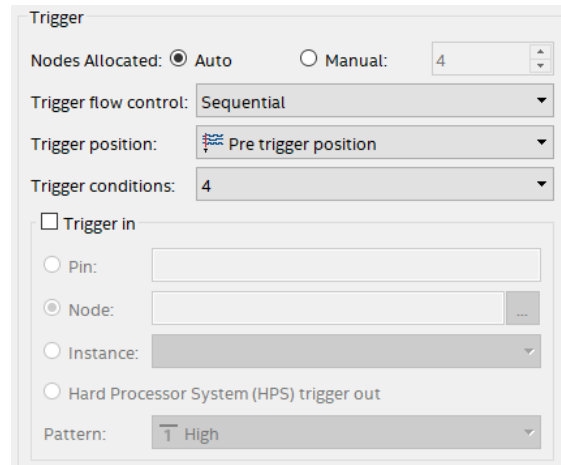


Figure 13. Set trigger conditions to 4.

- Right click the Trigger Condition 1 cell for KEY[0], and select Rising Edge. Do the same for the Trigger Condition 2 cell for KEY[1], Trigger Condition 3 for KEY[2], and Trigger Condition 4 for KEY[3]. You should end up with a window that looks like Figure 14.

Node		Data Enable	Trigger Enable	Trigger Conditions			
Type	Alias	Name					
		KEY[0]	4	4	1 Basic AND	2 Basic AND	3 Basic AND
		KEY[1]	4	4			4 Basic AND
		KEY[2]	4	4			
		KEY[3]	4	4			

Figure 14. Multiple trigger levels set.

- Now, recompile the design and load it onto the DE-series board again.
- Go back to the SignalTap window, click on the Data tab, and then click Processing > Run Analysis. Note that the window will say "Waiting for trigger" until the appropriate trigger condition is met. Then, in sequence, press and release keys 0, 1, 2, and then 3.

After this has been done, you will see the values of all the keys displayed as in Figure 15. Experiment by following the procedure outlined in this section to set up other trigger conditions and use the DE-series board to test these trigger conditions.

If you want to continuously probe the analyzer, instead of clicking "Run Analysis," click "Autorun Analysis" which is the icon right next to the "Run Analysis" icon. If you do this, every time the trigger condition is met the value in the display will be updated. You do not have to re-select "Run Analysis." To stop the "Autorun Analysis" function, click the  icon.

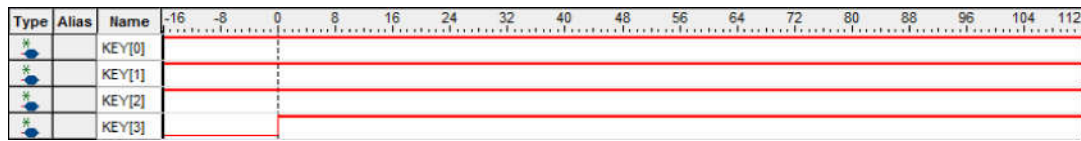


Figure 15. Logic Analyzer display when all four trigger conditions have been met.

6.2 Advanced Trigger Conditions

In this section we will learn how to create advanced trigger conditions. Our trigger condition will be whenever any one of the first 3 LED displays have a positive or negative edge. This means that the Logic Analyzer will update its display every time one of these inputs changes. Note that we could have any logical function of the nodes being probed to trigger the analyzer. This is just an example. After you implement this in the next few steps, experiment with your own advanced triggers.

1. Have the *keys* project opened and compiled from the previous examples in this tutorial.
2. Open the SignalTap window and select the Setup tab. In the Signal Configuration pane make sure that the number of Trigger Conditions is set to 1.
3. In the Trigger Conditions column of the node list, make sure the box is checked and select **Advanced** from the dropdown menu as in Figure 16. This will immediately bring up the window in Figure 17. This window allows you to create a logic circuit using the various nodes that you are probing with SignalTap.

Node			Data Enable	Trigger Enable	Trigger Conditions
Type	Alias	Name	4	4	1 Basic AND
*		KEY[0]	☒	☒	Basic AND
*		KEY[1]	☒	☒	Basic OR
*		KEY[2]	☒	☒	Advanced
*		KEY[3]	☒	☒	

Figure 16. Select Advanced from the Trigger Level dropdown menu.

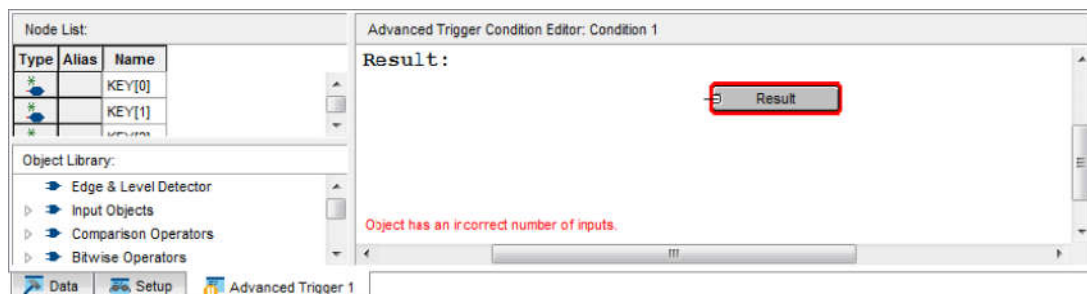


Figure 17. The Advanced Trigger editing window.

- In the node list section of this window, highlight the 3 nodes KEY[0] to KEY[2], and click and drag them into the white space of the Advanced trigger window, resulting in Figure 18. Note that you can also drag and drop each node individually.

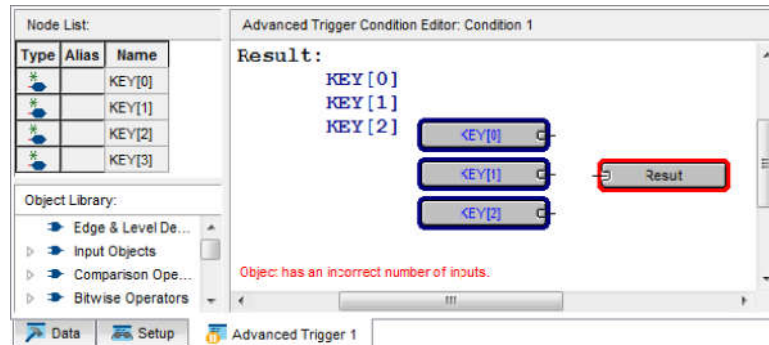


Figure 18. The three input nodes of interest dragged into the Advanced Trigger Editing Window.

- We now need to add the necessary logical operators to our circuit. We will need an OR gate as well as three edge level detectors. To access the OR gate, expand Logical Operators in the Object Library and select Logical Or, as in Figure 19. Then drag and drop the operator into the editing window.

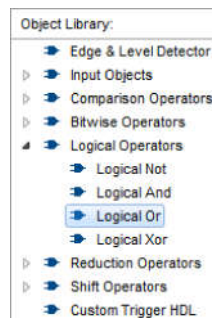


Figure 19. Select the Logical Or operator from the Object Library window and drag this into the editing window.

- In the object library click Edge and Level Detector and drag this into the editing window. Do this three times and then arrange the circuit as in Figure 20. The three inputs should each be connected to the input of an edge and level detector and the output of each of these detectors should be connected to the OR gate. The output of the OR gate should be connected to the output pin already in the editing window.

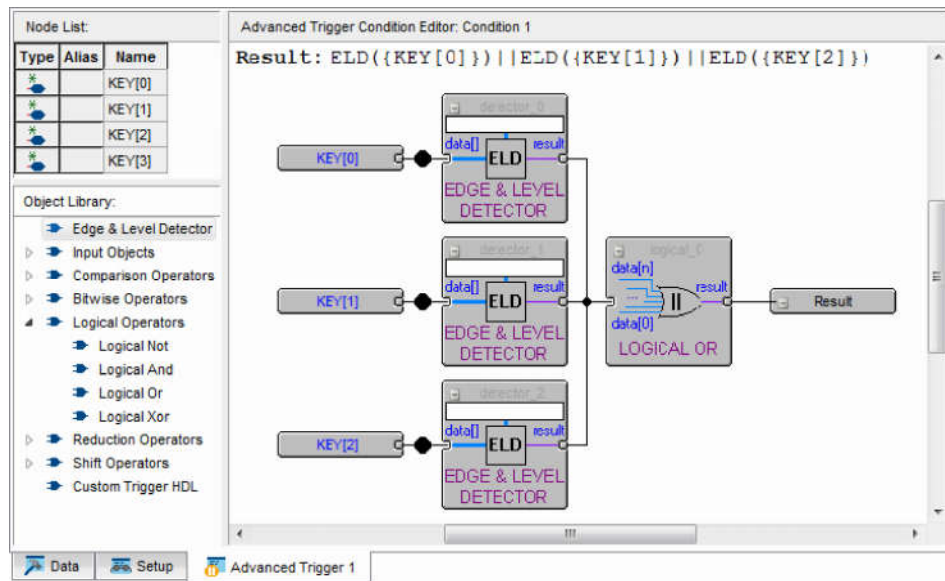


Figure 20. Arrange the elements to create a circuit that looks like this.

7. We now need to set each edge and level detector to sense either a falling edge or a rising edge. Double click one of the edge and level detectors, bringing up the window in Figure 21. Type E in the setting box and then click OK. This will mean that the detector will output 1 whenever there is either a falling edge or a rising edge of its input. Repeat this step for the two remaining edge and level detectors.

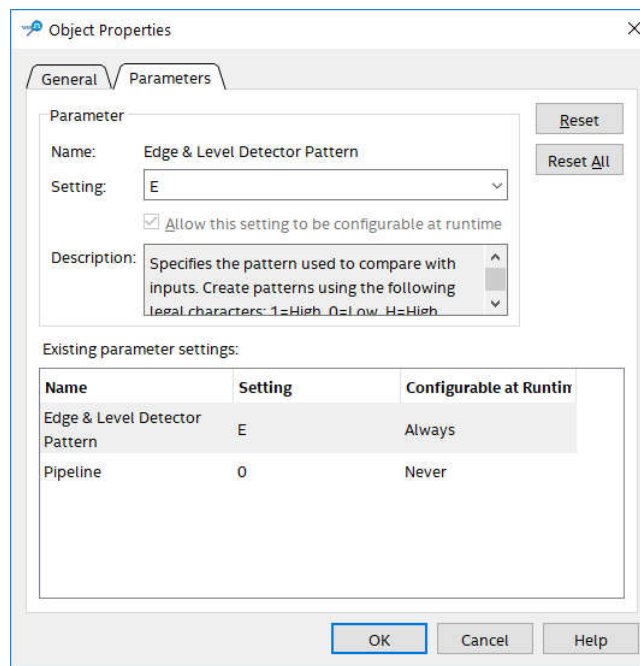


Figure 21. Type E in the setting box so that the function triggers on both rising and falling edges.

8. To test this Advanced trigger condition, compile the designed circuit again and load it onto the DE-series board. Then run Signal Tap as described in the previous section. You should note that the Analyzer should sense every time you change one of the first three keys on the board.

7 Sample Depth and Buffer Acquisition Modes

In this section, we will learn how to set the Sample Depth of our analyzer and about the two buffer acquisition modes. To do this, we will use the previous project and use segmented buffering. Segmented buffering allows us to divide the acquisition buffer into a number of separate, evenly sized segments. We will create a sample depth of 128 bits and divide this into eight 32-sample segments. This will allow us to capture 4 distinct events that occur around the time of our trigger.

1. Change the trigger condition back to Basic AND and have only one trigger condition. Make the trigger condition to be at the falling edge of KEY[0].
2. In the Signal Configuration pane of the SignalTap window, in the Sample depth dropdown menu of the Data pane select 128. This option allows you to specify how many samples will be taken around the triggers in your design. If you require many samples to debug your design, select a larger sample depth. Note, however, that if the sample depth selected is too large, there might not be enough room on the board to hold your design and the design will not compile. If this happens, try reducing the sample depth.

3. In the Signal Configuration pane of the SignalTap window, in the Data section of the pane check **Segmented**. In the dropdown menu beside Segmented, select **4 32 sample segments**. This will result in a pane that looks like Figure 22.

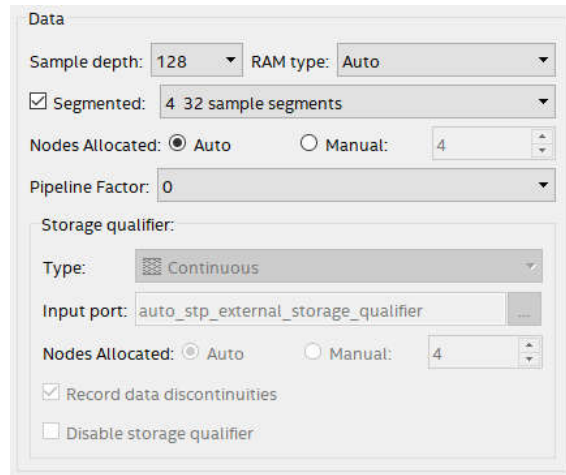


Figure 22. Select Segmented buffer acquisition mode with 4 32 sample segments.

4. Recompile and load the designed circuit onto the DE-series board. Now, we will be able to probe the design using the Segmented Acquisition mode.
5. Go back to the SignalTap window and click **Processing > Run Analysis**. Now, press and release KEY[0], and in between clicks change the values of the other 3 keys. After you have done this 4 times, the values in the buffer will be displayed in the data window, and this will display the values that the 4 keys had at around each trigger. A possible waveform is presented in Figure 23. This resulted from the user pressing and holding one more key between each click of KEY[0].

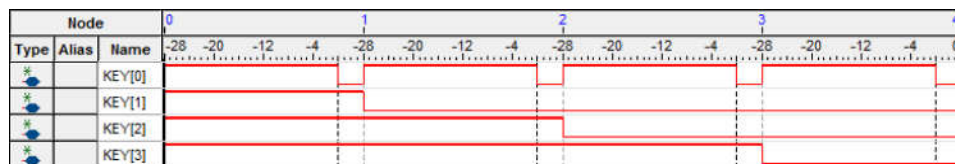


Figure 23. Possible waveforms that could result when using the Segmented Acquisition mode.

7.1 Use of Synthesis Keep Directive

Sometimes a design you create will have wires in it that the Quartus compiler will optimize away. A very simple example is the Verilog code below:

```

module threeInputAnd(SW, LEDR, CLOCK_50);
    input CLOCK_50;
    input [2:0] SW;
    output reg [0:0] LEDR;

    wire ab, abc /*synthesis keep*/;

    assign ab=SW[0]&SW[1];
    assign abc=ab&SW[2];

    always @ (posedge CLOCK_50)
    begin
        LEDR[0] <= abc;
    end
endmodule

```

Figure 24. Using the Synthesis Keep directive in Quartus Prime.

A diagram of this circuit is shown in Figure 25. The triangular symbols labeled **ab** and **abc** are buffers inserted by Quartus. They do not modify the signals passing through them.

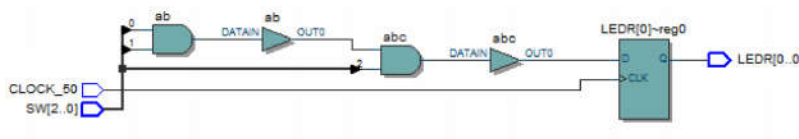


Figure 25. The circuit implemented by the code in Figure 24

We wish to instantiate a SignalTap module that will probe the values of the inputs SW[2:0] and the outputs LEDR[2:0]. We also want to probe the internal wire **ab**. However, normally when this Verilog code is compiled (without the */*synthesis keep*/* directive), the wire **ab** is optimized away into one logic element, as in Figure 26.

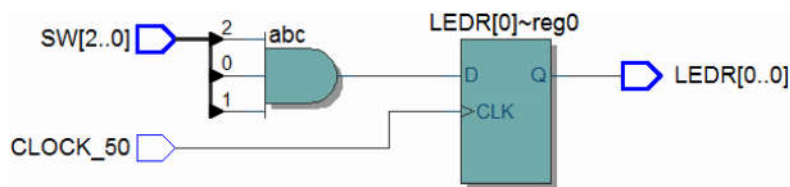


Figure 26. The same circuit without the Synthesis Keep directive.

If you wish to probe this internal wire, however, you will have to direct Quartus that you do not want this wire to be optimized away. To do so, place the text */*synthesis keep*/* on the line that declares the wire, right before the

semicolon of the line. Figure 24 already contains this directive. We will now demonstrate how this wire can be probed:

1. Create a new Quartus project `threeInputAnd` and copy the Verilog code from Figure 24. Compile the project.
2. Go to **Tools > SignalTap Logic Analyzer**, and then in the Setup pane of the SignalTap window, right click and choose **Add Nodes**.
3. For the **Filter** field, select **SignalTap: pre-synthesis**. Select `|threeInputAnd|` in the **Look in** drop-down menu and click the **List** button. Move the nodes **ab**, **SW[0]**, **SW[1]**, **SW[2]**, and **LEDR[0]** into the **Selected Nodes** list and then click **OK**.
4. In the **Signal Configuration** pane, select **CLOCK_50** as the clock signal.
5. Set a **Trigger Condition** to trigger when **ab** becomes high.
6. Import the relevant pin assignment file for the DE-series board (or assign the pins manually, as described in Section 7 of the Quartus Prime Introduction tutorials). For a DE1-SoC board, this file is named *DE1_SoC.qsf*.
7. Compile the project again.
8. Go to **Tools > Programmer** and load the circuit onto the DE-series board.
9. Open the SignalTap window again, and select the **Data** tab. Set all the switches on the DE-series board to the low position. Then, start the analysis by selecting **Processing > Run Analysis**.
10. Set the first two switches to the high position. The **Trigger Condition** should be satisfied.

Copyright © FPGAcademy.org. All rights reserved. FPGAcademy and the FPGAcademy logo are trademarks of FPGAcademy.org. This document is being provided on an “as-is” basis and as an accommodation and therefore all warranties, representations or guarantees of any kind (whether express, implied or statutory) including, without limitation, warranties of merchantability, non-infringement, or fitness for a particular purpose, are specifically disclaimed.

**Other names and brands may be claimed as the property of others.